
Finishing the $\forall$ PO-Free Fragment of $\mathcal{ALCI}_{\text{RCC5}}$

A fresh-occurrence cone-scheme proof strategy
and Lean certification blueprint

Purpose: detailed replacement plan for a complete decision proof
Input reviewed: round-two cold-attack package, 2026-08-28
Report date: 2026-08-28
Formal status: proof architecture and machine-checkable blueprint; Lean proof pending

Recommendation. Retire the finite-kernel and borrowed-edge extraction target. Decide the fragment by a finite greatest-fixed-point computation over *cone signatures*, then unfold a surviving finite strategy into an infinite fresh-occurrence forest. Proper-part edges are created only at birth; disconnection is the downward closure of designated horizontal birth cuts; overlap is residual and protected at its birth cut. This removes the three known obstructions by construction and yields a direct soundness/completeness theorem.

Abstract

This report gives a concrete route to a full Lean certification of decidability for the fragment of $\mathcal{ALCT}_{\text{RCC5}}$ in negation normal form whose closure contains no universal PO restriction. The route deliberately abandons finite quotient models, shared witness pools, and borrowed edges. Its finite object is instead a regular *strategy*: a cone signature consists of a local support type and a finite spectrum of possible strict-lower types. A monotone elimination operator retains precisely those signatures whose existential demands have compatible target signatures. The survivor set is finite and computable.

Soundness unfolds the strategy into fresh occurrences. Vertical birth edges generate a ranked strict order; DR births generate separated vertical components; PO births are protected as residual non-order, non-disjoint bridges. A one-way truth lemma realizes every selected type. Completeness maps any source model to its realized cone signatures: RCC5 singleton composition cells force the required PP, PPI, and DR transfer conditions, while the syntactic exclusion of $\forall$ PO removes every PO transfer obligation. Realized signatures form a post-fixed point and hence survive elimination.

The result is a finite decision procedure with a conservative double-exponential upper bound. The package accompanying this report contains the state rules, a Lean module map, a proof-obligation ledger, and executable structural checks. It is a formal certification plan, not a claim that the final Lean theorem already compiles.

Contents

1	Executive answer	1
2	Scope and exact semantic target	1
3	Why the target must be a regular strategy	2
4	Finite cone signatures	3
5	The finite greatest-fixed-point decider	5
6	Fresh-occurrence unfolding	5
7	Soundness: a survivor yields a model	7
8	Completeness: every model yields surviving signatures	8
9	Algorithm and conservative bound	9
10	Lean architecture and theorem graph	10
11	Regression suite	12
12	Milestones and go/no-go gates	13
13	Certification and release gates	14

14 Risk register	14
15 Relation to the literature	15
16 Final recommendation	15
A Proof-obligation dependency summary	16
B Package map	16
C Bibliography	16

1 Executive answer

No counterexample was found to the replacement architecture. It is a complete candidate soundness/completeness route: the finite state space and acceptance test are explicit, and every difficult obligation has a named Lean theorem. Occurrence geometry and the truth lemma remain genuine go/no-go research lemmas until they compile.

The final public theorem should state:

$$\forall C, \quad \text{ForallPOFree}(C) \longrightarrow \text{Decidable}(\exists M, x, M, x \models C).$$

It should be proved through a Boolean core

$$\text{coneSatCoreB}(C) = \text{true} \iff \exists M, x, M, x \models C.$$

under the displayed fragment hypothesis. A separate three-valued wrapper should report `outOfScope`, rather than `unsat`, when that hypothesis is absent.

The decisive change is to separate *finite control* from *model domain*. There need not be a finite model. A finite control state may recur indefinitely, but every existential use creates a fresh domain occurrence. Therefore a PP self-transition in the finite control graph denotes an infinite ascending chain, never a reflexive proper-part edge.

Issue in the previous route	Replacement invariant	Why it closes the issue
D1: cycles from borrowed order edges	Only fresh parent–child vertical birth edges; a height map increases on every base edge.	The transitive closure is irreflexive by an integer-rank argument.
D2: blocked source and target need incompatible order directions	A demand is realized by a new occurrence even when its target control state repeats.	Control-state equality never identifies domain points or reverses an old edge.
E: anchors create crossed PO/PP/DR constraints	No finite external anchors or globally shared witnesses exist.	Each recursive demand unfolds below its own occurrence; horizontal cuts are local.

2 Scope and exact semantic target

2.1 Syntax

The public wrapper accepts a raw concept and first proves semantic normalization:

$$M, x \models C \iff M, x \models \text{nnf}(C).$$

Inverse-role syntax is normalized at the same boundary by replacing r^- with the RCC5 converse atom. Define `ForallPOFree`(C) by inspecting the positive closure of `nnf`(C), not the surface syntax.

Let CI^+ be the positive subformula closure of `nnf`(C), including every modal body needed by structural induction. It is deliberately not completed with Boolean duals. Dual-complete types

would force preservation of negative PO information such as $\neg\exists\text{PO}.D = \forall\text{PO}.\neg D$, which the residual-PO construction does not support. Write

$$m = |\text{CI}^+|.$$

The role alphabet is

$$\text{At} = \{\text{EQ}, \text{PP}, \text{PPI}, \text{DR}, \text{PO}\}.$$

The input condition is

$$\forall D, \quad \forall \text{PO}.D \notin \text{CI}^+.$$

All other existential and universal role restrictions are permitted. Converse is already represented by PP/PPI; EQ is strong identity.

The theorem concerns *concept satisfiability only*: no TBoxes, ABoxes, nominals, counting restrictions, role conjunction, or identity-bearing assertions are included. The only roles after normalization are the five RCC5 atoms above.

2.2 Frames and models

The primary semantic interface is an abstract atomic RCC5 network: exactly one atom holds on each ordered pair, EQ is equality, converse is respected, and every triangle obeys the RCC5 composition table. The intended normal form is an ordered-disjoint frame:

- a strict partial order $<$ representing PP;
- a symmetric, irreflexive disjointness relation $\perp$;
- downward closure: $x' \leq x, y' \leq y, x \perp y$ imply $x' \perp y'$;
- PPI as converse, EQ as identity, and PO as the residual case.

The existing or separately proved normal-form theorem then supplies the entire RCC5 composition table. For a self-contained semantic endpoint, Section 6.6 gives an exact representation by arbitrary nonempty sets. It does not establish regular-closed or topological region representability.

2.3 What this report does and does not certify

The report specifies the definitions, intermediate lemmas, theorem dependency order, finite bounds, regression obligations, and release gates needed for a final Lean certificate. The included scripts validate consistency of the blueprint and replay the known finite adversarial instances. They do not replace the missing Lean sources. No **sorry-free** theorem is claimed until the module graph in Section 10 compiles and the audit gates in Section 13 pass.

3 Why the target must be a regular strategy

3.1 The fragment lacks the finite model property

Let

$$H = \exists\text{PP}.\top, \quad C_\infty = H \wedge \forall\text{PP}.H.$$

This concept is $\forall\text{PO}$ -free. It is satisfiable on the strict chain of finite initial segments

$$\{0\} \text{ PP } \{0, 1\} \text{ PP } \{0, 1, 2\} \text{ PP } \dots$$

But no finite strict transitive model can satisfy it. From a root satisfying H choose a proper superpart; the universal conjunct ensures that each proper superpart again has a proper superpart. Finiteness would force repetition, contradicting irreflexivity.

Therefore no proof based on a bounded finite quotient domain can be complete. The finite certificate must represent an infinite regular model, exactly as a finite automaton represents infinitely many runs.

3.2 Control recurrence is not point identification

Suppose a finite state q has a PP transition to itself. The unfolding has occurrences

$$\epsilon, \quad d, \quad dd, \quad ddd, \dots$$

all labelled by q , with strictly increasing height. This is sound. Identifying them would instead assert $xPPx$, which is impossible. The Lean development must enforce this distinction at the type level: occurrences are finite demand paths, while control states are labels attached to paths.

3.3 Why no parity condition is needed

ALCI has no least or greatest fixpoint operators. Every existential demand is fulfilled at the next generation, and universal restrictions are safety invariants checked along the generated relations. Infinite recurrence therefore carries no fairness or eventuality obligation. The elimination procedure is a one-priority safety game, not a parity game.

4 Finite cone signatures

4.1 Support types

A support type $T \subseteq \text{CI}^+$ is locally Hintikka-consistent:

- $\perp \notin T$, and $\top \in T$ whenever $\top \in \text{CI}^+$;
- complementary concept literals do not both occur;
- $D \wedge E \in T$ implies $D, E \in T$;
- $D \vee E \in T$ implies $D \in T$ or $E \in T$;

Types are support sets, not maximal or Boolean-complete theories. The semantic induction proves only membership-to-truth; no converse truth lemma is needed.

Let Ty be the finite set of support types and let

$$N = |\text{Ty}| \leq 2^m.$$

For $r \in \text{At}$ define the universal body set

$$A_r(T) = \{D \in \text{CI}^+ \mid \forall r. D \in T\}.$$

4.2 State definition

A cone signature is a pair

$$q = (T, S), \quad T \in \text{Ty}, \quad S \subseteq \text{Ty}.$$

Its inclusive cone is

$$\text{cone}(q) = S \cup \{T\}.$$

The intended invariant is that every strict lower occurrence of a point labelled by q has a support type in S . The spectrum is an upper bound, not an exact list; this monotonicity is important in soundness.

The number of raw signatures is

$$K = N 2^N \leq 2^{m+2^m}.$$

4.3 Static local admissibility

A state $q = (T, S)$ is statically admissible when:

1. T is a support type;
2. $\forall \text{EQ}.D \in T$ implies $D \in T$;
3. $\exists \text{EQ}.D \in T$ implies $D \in T$;
4. for every $U \in S$,

$$A_{\text{PP}}(U) \subseteq T \quad \text{and} \quad A_{\text{PPI}}(T) \subseteq U.$$

The fourth condition is the entire vertical universal check. If a point of type U is a proper part of a point of type T , then $\forall \text{PP}$ bodies at the lower point and $\forall \text{PPI}$ bodies at the upper point must hold at the other endpoint.

4.4 Transition compatibility

Let $\delta = \exists r.D \in T_q$ be a demand at state q , and let q' be a proposed target. First require $D \in T_{q'}$. The role-specific conditions are:

Role	Additional compatibility condition
EQ	No child is created; require $D \in T_q$.
PP	$\text{cone}(q) \subseteq S_{q'}$.
PPI	$\text{cone}(q') \subseteq S_q$.
DR	For all $U \in \text{cone}(q)$ and $V \in \text{cone}(q')$, $A_{\text{DR}}(U) \subseteq V$ and $A_{\text{DR}}(V) \subseteq U$.
PO	No additional transfer condition.

The PO row is not a shortcut. There is no formula $\forall \text{PO}.E$ in the closure, so a PO edge carries no universal body. The unfolding must still guarantee that the designated endpoints are exactly PO; that is a geometric theorem, not a type-transfer condition.

5 The finite greatest-fixed-point decider

Let **Static** be the finite set of statically admissible signatures. For $X \subseteq \mathbf{Static}$ define

$$\Phi(X) = \left\{ q \in X \mid \begin{array}{l} \text{for every non-EQ existential demand } \delta \text{ in } T_q, \\ \text{there exists } q' \in X \text{ compatible with } (q, \delta), \text{ and} \\ \text{every EQ demand is locally satisfied} \end{array} \right\}.$$

This operator is monotone and reductive. Compute

$$X_0 = \mathbf{Static}, \quad X_{i+1} = \Phi(X_i).$$

The sequence descends and stabilizes in at most $|\mathbf{Static}| \leq K$ strict iterations. Let X_∞ be the stable set.

Acceptance criterion. The raw input C_0 is accepted iff some $q \in X_\infty$ contains the prepared closure index of $\text{nnf}(C_0)$.

Finite certificate. For every survivor q and every non-EQ demand in T_q , choose one compatible target in X_∞ . The resulting strategy has at most K states and at most mK chosen transitions. The abstract scheme uses a proof-independent total target function from state and full role-aware demand code to state; membership and correctness are separate fields.

```
def coneSatCoreB (C : Concept) : Bool :=
  let cl      := positiveClosure (nnf C)
  let states  := enumerateStaticStates cl
  let survive := iterateAtMost states.card (eliminate cl) states
  survive.any fun q => q.ty.contains (nnf C)

theorem coneSatCoreB_correct (hfree : ForallPOFree C) :
  coneSatCoreB C = true <-> Satisfiable C

def coneCheck (C : Concept) : CheckResult :=
  if forallPOFreeB C then
    if coneSatCoreB C then .sat else .unsat
  else .outOfScope

theorem coneCheck_sat_iff :
  coneCheck C = .sat <->
    ForallPOFree C /\ Satisfiable C
theorem coneCheck_unsat_iff :
  coneCheck C = .unsat <->
    ForallPOFree C /\ Not (Satisfiable C)
theorem coneCheck_outOfScope_iff :
  coneCheck C = .outOfScope <-> Not (ForallPOFree C)
```

The proof of `coneSatCoreB_correct` is split into the soundness and completeness constructions below. The executable procedure must use reflected Boolean predicates with correctness theorems, rather than classical existence tests hidden behind opaque decidability.

6 Fresh-occurrence unfolding

6.1 Occurrence domain

Fix a chosen transition function for the survivors. An occurrence is a finite valid word of demand choices. The root is the empty word. If occurrence x has control state q and δ is a non-EQ demand in T_q , then $x \frown \delta$ is a distinct child labelled by the selected target state.

Two occurrences with the same control state remain different domain elements. Each non-EQ demand at each occurrence receives its own fresh child. A transition pointer is therefore only a control pointer; it is never a semantic backpointer.

6.2 Vertical birth order and rank

Orient base order edges according to the demand role:

- a PP child lies above its parent: $x <_0 x\delta$;
- a PPI child lies below its parent: $x\delta <_0 x$;
- DR and PO children start new vertical components;
- EQ creates no child.

Let $<$ be the transitive closure of $<_0$. Define a height $h : \text{Occurrence} \rightarrow \mathbb{Z}$ by adding 1 at a PP birth, subtracting 1 at a PPI birth, and resetting the relative origin at a horizontal birth. Every oriented base edge satisfies

$$x <_0 y \implies h(x) + 1 = h(y).$$

Thus every nonempty $<_0$ path strictly raises height, and $<$ is irreflexive. Transitivity holds by definition. Parent uniqueness of words gives the induction scheme needed for this rank proof.

6.3 Vertical components

Give each occurrence a component tag equal to the sequence of uniquely identified horizontal birth edges (DR or PO, including the child occurrence) on its ancestry. Vertical births preserve the tag; horizontal births append a fresh edge identifier. Order edges never cross component tags. Removing horizontal births yields vertical components, and contracting each such component yields a tree whose edges are the unique horizontal birth bridges.

This component tree is the geometric skeleton. It replaces every global anchor, witness pool, and kernel-incidence construction.

6.4 Disjointness closure

For each DR birth bridge (x, y) , declare the two inclusive vertical cones at that bridge disjoint. Equivalently, take the symmetric downward closure of the seed bridges:

$$u \perp v \iff \exists (x, y) \in \text{DRSeed}, (u \leq x \wedge v \leq y) \vee (u \leq y \wedge v \leq x).$$

Here $\leq$ is the reflexive closure of $<$.

The central separator lemma states that the two sides of a DR birth cut have no common lower occurrence. It follows by unique component ancestry: any common lower would have to cross the unique horizontal bridge, but order never crosses component tags. Hence $\perp$ is irreflexive. Symmetry and downward closure are immediate, and

$$x < y \implies \neg(x \perp y).$$

6.5 Protected overlap

Define RCC5 atoms from the order and disjointness normal form:

$$R(x, y) = \begin{cases} \text{EQ} & x = y, \\ \text{PP} & x < y, \\ \text{PPI} & y < x, \\ \text{DR} & x \perp y, \\ \text{PO} & \text{otherwise.} \end{cases}$$

At a PO birth bridge, the endpoint occurrences differ, lie in different vertical components, and are not seeded disjoint. Unique-bridge separation prevents any later vertical path between them; downward DR closure cannot cross a non-DR bridge. Therefore the birth endpoints remain neither ordered nor disjoint and hence are exactly PO. This is the *protected-PO theorem*.

It is important to prove this theorem from component ancestry before defining PO as residual. An incremental default-PO labelling algorithm would be unsound because later order or disjointness closure could overwrite an early guess.

6.6 Exact set representation

If the public semantics require literal nonempty sets, represent the ordered-disjoint frame without quotient types. Let

$$\text{Tok} = \{(a, b) \mid \neg(a \perp b)\}$$

and define

$$\mathcal{R}_x = \{(a, b) \in \text{Tok} \mid a \leq x \vee b \leq x\}.$$

Then:

$$\mathcal{R}_x \subseteq \mathcal{R}_y \iff x \leq y, \quad \mathcal{R}_x \cap \mathcal{R}_y = \emptyset \iff x \perp y.$$

The self token (x, x) makes $\mathcal{R}_x$ nonempty and distinguishes incomparable points. Ordered pairs are a useful proof-engineering choice: they avoid quotient soundness obligations and retain both sides of a non-disjoint witness. These equivalences transfer the ordered-disjoint atom exactly to standard RCC5 relations on nonempty sets.

7 Soundness: a survivor yields a model

7.1 Spectrum invariant

Let $\sigma(x)$ be the control state at occurrence x . Prove simultaneously with the unfolding that

$$y < x \implies T_{\sigma(y)} \in S_{\sigma(x)}.$$

For one vertical birth this is exactly the PP or PPI transition rule. For a transitive path, the inclusive-cone subset propagates the earlier endpoint type into the later spectrum. This is why the transition rules use $\text{cone}(q)$ rather than only S_q .

7.2 Universal restrictions

Assume $D \in T_{\sigma(x)}$ is a universal body obligation.

- If $\forall \text{PP}.D \in T_{\sigma(x)}$ and $x < y$, the spectrum invariant and static local admissibility put D in $T_{\sigma(y)}$.
- If $\forall \text{PPI}.D \in T_{\sigma(x)}$ and $y < x$, use the converse half of the same local rule.
- If $\forall \text{DR}.D \in T_{\sigma(x)}$ and $x \perp y$, choose the originating DR birth cut. Both endpoint types lie in the inclusive cones of its control states. The cross-cone DR compatibility condition puts D in $T_{\sigma(y)}$.
- EQ is local.
- The PO case is impossible by the fragment predicate.

7.3 Existential restrictions

For every non-EQ demand $\exists r.D$ at x , the chosen strategy creates a child y whose type contains D . The geometry proves that $R(x, y) = r$: this is immediate for PP and PPI, the seed definition for DR, and protected overlap for PO. An EQ demand is realized by x itself.

7.4 One-way truth lemma

Interpret each concept name P by

$$P^M = \{x \mid P \in T_{\sigma(x)}\}.$$

Induction on $D \in \text{CI}^+$ proves

$$D \in T_{\sigma(x)} \implies M, x \models D.$$

Only this direction is required. Boolean cases follow from Hintikka consistency; existential and universal cases use the preceding lemmas. If the accepted root signature contains the prepared index of $\text{nnf}(C_0)$, the root satisfies the normalized concept; `sat_nnf_iff` transfers this back to C_0 .

Soundness theorem.

$$\text{ForallPOFree}(C_0) \wedge \text{coneSatCoreB}(C_0) = \text{true} \implies \exists M, x, M, x \models C_0.$$

8 Completeness: every model yields surviving signatures

Let M be any model and x a point. Define its realized support type and lower spectrum:

$$T_x = \{D \in \text{CI}^+ \mid M, x \models D\}, \quad S_x = \{T_y \mid y \text{PP} x\}.$$

Since Ty is finite, S_x is a finite subset of Ty even when the model has infinitely many points. Put $q_x = (T_x, S_x)$.

8.1 Static admissibility

The truth definition gives local Hintikka consistency and the EQ conditions. If $U = T_y \in S_x$, then $y \text{PP} x$. Every $\forall \text{PP}$ obligation at y holds at x , and every $\forall \text{PPI}$ obligation at x holds at y . Thus q_x is statically admissible.

8.2 Source transitions

Suppose y witnesses an existential demand at x .

- If $x\text{PP}y$, then every $z \leq x$ is a strict lower point of y , including x . Therefore $\text{cone}(q_x) \subseteq S_y$.
- If $x\text{PPI}y$, the dual inclusion $\text{cone}(q_y) \subseteq S_x$ holds.
- If $x\text{DR}y$, then every $z \leq x$ and $w \leq y$ is disjoint. This follows from the RCC5 singleton cells

$$\text{comp}(\text{PP}, \text{DR}) = \{\text{DR}\}, \quad \text{comp}(\text{DR}, \text{PPI}) = \{\text{DR}\}.$$

Hence every DR universal body transfers across all pairs in $\text{cone}(q_x) \times \text{cone}(q_y)$.

- If $x\text{PO}y$, the target body holds at y and no universal PO transfer exists.
- If $x\text{EQ}y$, then $x = y$ and the demand is local.

Thus every source witness induces a compatible transition between realized signatures.

8.3 Post-fixed-point argument

Let

$$R_M = \{q_x \mid x \in M\} \subseteq \text{Static}.$$

In Lean, R_M is a proof-level **Set ConeState**, defined by an existential source point; it is not a computable **Finset** over an arbitrary domain. Every existential demand at every state in R_M has a target again in R_M , so

$$R_M \subseteq \Phi(R_M).$$

For a monotone operator on a finite powerset, every post-fixed set lies below the greatest fixed point. Therefore $R_M \subseteq X_\infty$. If $M, x_0 \models C_0$, semantic NNF correctness gives $\text{nnf}(C_0) \in T_{x_0}$, so an accepting survivor exists.

Completeness theorem.

$$\text{ForallPOFree}(C_0) \wedge (\exists M, x, M, x \models C_0) \implies \text{coneSatCoreB}(C_0) = \text{true}.$$

This proof uses neither a finite model property nor a patchwork theorem. It compresses only the *local source signature*, not the source domain or its witnesses.

9 Algorithm and conservative bound

With $N \leq 2^m$ and $K = N2^N$, enumerate all K raw signatures, filter static admissibility, precompute compatible transition triples, and iterate elimination for at most K rounds.

A deliberately conservative implementation bound is:

$$O(K^2 N^2 m) \quad \text{for compatibility precomputation,}$$

$$O(K^3 m) \quad \text{for naive elimination.}$$

Since $K \leq 2^{m+2^m}$, this gives a deterministic

$$2^{2^{O(m)}}$$

time upper bound. This is an upper bound for the proposed implementation, not a claimed tight complexity classification. The release theorem should state only decidability until a Lean cost model and a separately reviewed complexity proof are available.

Several engineering improvements are safe but should follow correctness: bit-vector types, indexed demand buckets, reverse dependency lists, and work-list elimination.

10 Lean architecture and theorem graph

10.1 Modules

Module	Responsibility
RCC5/Atom.lean	atoms, converse, finite enumeration
RCC5/CompTable.lean	composition table and singleton-cell lemmas
RCC5/Network.lean	abstract strong RCC5 networks
RCC5/OrderedDisjoint.lean	normal form and composition closure
RCC5/TokenRepresentation.lean	exact region-set representation
Logic/Syntax.lean	raw concepts and role restrictions
Logic/NNF.lean	semantic NNF correctness and inverse-role normalization
Logic/Prepared.lean	normalized root code and positive closure package
Logic/PositiveClosure.lean	positive subformula closure and finiteness
Logic/POFree.lean	syntactic predicate and reflection
Logic/Semantics.lean	abstract and concrete model satisfaction
Logic/SupportType.lean	Hintikka types, enumeration, reflection
Logic/Demand.lean	finite existential demands
ConeScheme/State.lean	signatures, spectra, inclusive cones
ConeScheme/StateEnum.lean	exhaustive finite enumeration
ConeScheme/LocalRules.lean	static admissibility and Boolean checker
ConeScheme/Transition.lean	five role-specific compatibility rules
ConeScheme/Scheme.lean	abstract valid scheme with proof-independent targets
ConeScheme/Occurrence.lean	fresh demand-word domain
ConeScheme/VerticalComponent.lean	component tags and bridge uniqueness
ConeScheme/BaseOrder.lean	height, transitive closure, strict order
ConeScheme/DisjointClosure.lean	DR cuts and downward closure
ConeScheme/ProtectedPO.lean	exact residual overlap at PO births
ConeScheme/Frame.lean	ordered-disjoint and RCC5 frames
ConeScheme/Model.lean	valuation from support labels
ConeScheme/SpectrumInvariant.lean	actual lower types lie in spectra
ConeScheme/TruthLemma.lean	one-way semantic induction
ConeScheme/SchemeSoundness.lean	every valid abstract scheme has a model
ConeScheme/FiniteGfp.lean	generic reductive finite fixed-point facts
ConeScheme/Elimination.lean	reflected viability and survivor computation
ConeScheme/StrategyExtraction.lean	accepting survivors yield an abstract scheme
ConeScheme/SourceSignature.lean	signatures extracted from arbitrary models
ConeScheme/SourceTransitions.lean	source witness compatibility
ConeScheme/Soundness.lean	accepted strategy unfolds to a model
ConeScheme/Completeness.lean	realized signatures survive

Module	Responsibility
ConeScheme/Decision.lean	public Boolean correctness and decidability
Regression/*.lean	adversarial instances and mutation guards
Audit/PublicTheorems.lean	only supported public exports
Audit/AxiomReport.lean	axiom checks for exported theorems
Audit/ExecutableDeps.lean	executable import and code-generation checks

The former multi-tier extraction should move below `Legacy/MultiTier/`. The public decision theorem must not import anything from `Legacy`.

10.2 Core declarations

```

structure Prepared (C : Concept) where
  nnfC      : NNFCConcept
  closure   : Finset NNFCConcept
  rootIx    : ClosureIx closure
  nnf_eq    : nnfC = nnf C
  cl_eq     : closure = positiveClosure nnfC
  root_eq   : decode rootIx = nnfC

abbrev TypeCode (p : Prepared C) := Finset (ClosureIx p.closure)

structure ConeState (p : Prepared C) where
  ty        : TypeCode p
  lower     : Finset (TypeCode p)

def ConeState.cone (q : ConeState p) : Finset (TypeCode p) :=
  insert q.ty q.lower

abbrev DemandCode (p : Prepared C) := ClosureIx p.closure
def LocalOK (q : ConeState p) : Prop := ...
def StepOK (q : ConeState p) (d : DemandCode p)
  (q' : ConeState p) : Prop := ...

structure ConeScheme (p : Prepared C) where
  live      : Finset (ConeState p)
  root      : ConeState p
  root_mem  : root ∈ live
  root_has  : p.rootIx ∈ root.ty
  next      : ConeState p -> DemandCode p -> ConeState p
  next_mem  : ...
  next_ok   : ...

def prune (X : Finset (ConeState p)) :=
  X.filter fun q => viableInB p X q
def survivors (p : Prepared C) : Finset (ConeState p) := ...

theorem regular_forest_model_iff
  (p : Prepared C) (hfree : ForallPOFree C) :
  (exists q, q ∈ survivors p /\ p.rootIx ∈ q.ty) <=>
    Satisfiable C

theorem satisfiableDecidable (hfree : ForallPOFree C) :
  Decidable (Satisfiable C) :=
  if h : coneSatCoreB C = true then

```

```

    isTrue ((coneSatCoreB_correct hfree).mp h)
  else
    isFalse (fun hs => h ((coneSatCoreB_correct hfree).mpr hs))

```

10.3 Named proof obligations

The critical theorem spine is:

1. `sat_nnf_iff`, `inverseRole_nnf_conv`, and normalized-root preparation;
2. Boolean reflection:
 - `forallPOFreeB_reflect`;
 - `supportTypeOKB_reflect`;
 - `stateLocalOKB_reflect`;
 - `demandsB_complete`;
 - `transitionOKB_reflect`;
 - `hasTargetB_reflect`;
 - `viableInB_reflect`;
 - `prune_mem_iff`;
 - `acceptingB_reflect`.
3. `survivors_fixed`, `postfixed_subset_survivors`, and `survivors_to_scheme`;
4. `child_unique_parent`, `baseOrder_raises_height`, and `unfold_lt_rank`;
5. Geometry and representation:
 - `dr_cut_separates`;
 - `occurrence_orderedDisjoint`;
 - `orderedDisjoint_rcc5`;
 - `tokenRepresentation_exact`.
6. `protectedPO_residual`;
7. `actualLower_mem_spectrum`;
8. `truth_lemma` and `scheme_sound`, both carrying the normalized-closure fragment hypothesis;
9. `sourceSig_localOK`, the five `source_role_step_ok` lemmas, and `sourceSig_survives`;
10. `regular_forest_model_iff`, `coneSatCoreB_correct`, and `satisfiableDecidable`.

The bundled JSON ledger records each obligation with dependencies, evidence status, and its release gate. CI should reject a theorem marked complete when a dependency is not complete.

11 Regression suite

The proof should be developed against semantic regressions, not only positive examples.

Regression	Required behavior	Mutation caught
No finite model	Accept C_∞ via recurrent control and fresh occurrences.	Identifying recurrent states with points.
Mixed vertical cycle	Accept recurrent mixed control patterns while proving the unfolded occurrence order acyclic.	Identifying control states or reversing PPI orientation.
Protected PO	A PO demand is exactly residual at its birth bridge.	Labelling PO before closure.
DR downward	Every pair below opposite DR sides remains DR.	Checking only seed endpoints.
Role-aware demand	Identical target concepts under different roles remain distinct.	Indexing witnesses only by concept body.
D1 multi-group cycle	Replays the known borrowed-edge cycle attack.	Reintroducing shared borrowed vertical edges.
D2 seven-point	Replays the incompatible blocked-source/target instance.	Identifying target occurrences with old points.
E anchor compatibility	Replays the crossed-anchor incompatibility.	Adding global anchors or shared horizontal witnesses.

For every successful theorem, add at least one mutation test that deletes or weakens its decisive hypothesis. In particular, the DR completeness proof must fail if either singleton composition cell is removed, and protected overlap must fail if component bridge uniqueness is removed.

12 Milestones and go/no-go gates

Gate	Focus	Exit criterion	Estimate
G0	Source recovery	Existing syntax, semantics, RCC5 table, and theorem names compile.	1–2 weeks
G1	Finite control	Enumeration, reflected checks, GFP, and certificate extraction compile.	2–3 weeks
G2	Occurrence geometry	Strict order, DR separator, protected PO, and normal form compile.	3–5 weeks
G3	Soundness	Spectrum invariant and truth lemma compile with no admitted facts.	3–4 weeks
G4	Completeness	Source signatures are a post-fixed point for every model.	2–4 weeks
G5	Public theorem	Boolean correctness and Decidable instance compile.	2–3 weeks
G6	Audit and release	Dual clean builds, regressions, axiom report, and package manifest pass.	2–3 weeks

A realistic estimate is four to six focused months after source recovery. G2 is the first hard go/no-go gate. Do not invest in optimized enumeration or complexity certification until `protectedPO_residual` and `occurrence_orderedDisjoint` compile. If G2 fails, geometry is

the first suspected fault. A counterexample may nevertheless show that the finite signature or transition predicate is too weak and require a redesign of G1.

13 Certification and release gates

13.1 Logical hygiene

The release build must:

- contain no `sorry`, `admit`, `sorryAx`, or new axioms;
- contain no unsafe declaration in the proof dependency cone;
- compare `#print axioms` for every public theorem against an exact allowlist, and separately scan all declarations for unused custom axioms;
- keep classical choice explicit and outside the Boolean decider;
- use kernel `by decide` for release regressions, or explicitly account for any extra trust introduced by `native_decide`;
- use warnings as errors;
- prove Boolean reflection for all finite predicates used by the executable.

13.2 Dependency hygiene

Build the project twice: once with the full tree and once from a minimal public import that excludes `Legacy`. Generate the transitive import graph and fail CI if the executable finite-control modules reach `SourceSignature`, `Completeness`, or `Legacy`. Run a generated-code evaluation test for `coneCheck`. Pin the Lean toolchain and all Lake dependencies in the package manifest.

13.3 Reproducibility

The release archive should include:

- exact Lean and Lake versions;
- a clean-build script;
- the theorem/axiom report;
- all regression files;
- a SHA-256 manifest;
- this report and the machine-readable obligation ledger.

Run the clean build on two independent runners. Extract the archive into a new directory before the final build, so no untracked local file can mask a missing dependency.

14 Risk register

Risk	Mitigation	Decision point
Positive closure omits a needed body	Prove structural closure lemmas before defining support types; add a theorem that every semantic induction child belongs to CI^+ .	G1
Spectrum transitivity is underspecified	State the invariant with inclusive cones and prove one-edge propagation before transitive closure.	G3
PPI height orientation is reversed	Centralize role orientation in one definition and test mixed PP/PPI words.	G2
DR closure becomes reflexive	Prove the cut separator before defining the ordered-disjoint instance.	G2
PO birth is later ordered or disjoint	Use component tags and unique bridges; prove protected overlap as a standalone theorem.	G2
Abstract and concrete semantics diverge	Complete the ordered-token representation and prove atom-by-atom equivalence.	G2/G3
Classical proof is mistaken for executable code	Require reflected Booleans and inspect generated evaluator dependencies.	G1/G5
Complexity claim outruns certification	Publish decidability first; keep the double-exponential estimate explicitly conservative.	G6

15 Relation to the literature

The plan uses standard ideas but the central combination is bespoke. RCC5 and its composition calculus originate with Randell, Cui, and Cohn [1]; formal model and composition-table issues are analyzed by Li and Ying [3], while tractable RCC5 sublanguages were classified by Jonsson and Drakengren [2]. Type elimination and tableau methods for description logics motivate the finite control layer, including Donini and Massacci [4] and the treatment of inverse and transitive roles by Horrocks and Sattler [5]. Regular grammar logics with converse provide nearby automata and tableau methodology [6, 7]; Fischer–Ladner closure and automata-theoretic modal decision procedures are further background [8, 9]. Lean 4 is described by de Moura and Ullrich [10].

None of these sources proves the theorem targeted here. In particular, the fresh-occurrence cone scheme, the spectrum transition conditions, and the protected-PO regular-forest theorem require an original proof. The literature supports the surrounding methods and RCC5 semantics, not the transfer step.

16 Final recommendation

Proceed with the cone-scheme architecture and freeze the previous multi-tier development as legacy evidence. Implement G1 and G2 first. Their interfaces are narrow enough that two proof engineers can work independently: one on finite reflected control, one on fresh-occurrence geometry. Join them only at the spectrum invariant.

The project should be declared complete only when:

1. the regular-forest model equivalence is proved with the exact fragment predicate;

2. the executable Boolean decider is connected to that equivalence;
3. the public theorem has an acceptable axiom report;
4. every adversarial regression passes in a clean extracted archive.

This is a materially different proof, not a patch to the blocked extraction. Its main advantage is conceptual: the finite object records *how to generate* a model, while the infinite unfolding supplies the fresh geometry that RCC5 proper-part chains require.

A Proof-obligation dependency summary

Layer	Inputs	Output theorem
Reflection	syntax, finite sets	exact Boolean predicates
Finite GFP	reflected local and transition checks	survivor fixed point
Geometry	chosen survivor strategy	ordered-disjoint occurrence frame
Soundness	geometry, spectrum invariant	accepted implies satisfiable
Source map	arbitrary model, singleton RCC5 cells	realized signatures post-fixed
Completeness	source map, greatest fixed point	satisfiable implies accepted
Decision	finite enumeration, equivalence	full decidability

The machine-readable file `blueprint/PROOF_OBLIGATIONS.json` is authoritative for individual names and dependencies. Status values in that file deliberately distinguish paper argument, executable check, inherited result, and pending Lean proof.

B Package map

- `report/`: this source report and compiled PDF;
- `blueprint/DECISION_PROCEDURE.md`: concise mathematical specification;
- `blueprint/LEAN_MODULE_MAP.md`: module ownership and import order;
- `blueprint/PROOF_OBLIGATIONS.json`: theorem dependency ledger;
- `blueprint/check_blueprint.py`: structural ledger validation;
- `regressions/`: finite adversarial probes from the round-two attack;
- `run_checks.sh`: one-command blueprint and regression replay.

C Bibliography

References

- [1] D. A. Randell, Z. Cui, and A. G. Cohn. A spatial logic based on regions and connection. In *Principles of Knowledge Representation and Reasoning: KR'92*, pages 165–176, 1992. <https://cdn.aaai.org/KR/1992/KR92-017.pdf>.

- [2] P. Jonsson and T. Drakengren. A complete classification of tractability in RCC-5. *Journal of Artificial Intelligence Research*, 6:211–221, 1997. <https://doi.org/10.1613/jair.385>.
- [3] S. Li and M. Ying. Region Connection Calculus: its models and composition table. *Artificial Intelligence*, 145:121–146, 2003. [https://doi.org/10.1016/S0004-3702\(02\)00372-7](https://doi.org/10.1016/S0004-3702(02)00372-7).
- [4] F. M. Donini and F. Massacci. EXPTIME tableaux for ALC. *Artificial Intelligence*, 124:87–138, 2000. [https://doi.org/10.1016/S0004-3702\(00\)00070-9](https://doi.org/10.1016/S0004-3702(00)00070-9).
- [5] I. Horrocks and U. Sattler. A description logic with transitive and inverse roles and role hierarchies. *Journal of Logic and Computation*, 9(3):385–410, 1999. <https://doi.org/10.1093/logcom/9.3.385>.
- [6] S. Demri and H. de Nivelle. Deciding regular grammar logics with converse through first-order logic. *Journal of Logic, Language and Information*, 14:289–329, 2005. <https://arxiv.org/abs/cs/0306117>.
- [7] L. A. Nguyen. A tableau calculus for regular grammar logics with converse. In *TABLEAUX 2007*, LNCS 4548, pages 421–436. https://doi.org/10.1007/978-3-540-73099-6_29.
- [8] M. J. Fischer and R. E. Ladner. Propositional dynamic logic of regular programs. *Journal of Computer and System Sciences*, 18:194–211, 1979. [https://doi.org/10.1016/0022-0000\(79\)90046-1](https://doi.org/10.1016/0022-0000(79)90046-1).
- [9] M. Y. Vardi and P. Wolper. Automata-theoretic techniques for modal logics of programs. *Journal of Computer and System Sciences*, 32:183–221, 1986. [https://doi.org/10.1016/0022-0000\(86\)90026-7](https://doi.org/10.1016/0022-0000(86)90026-7).
- [10] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE 28*, 2021. <https://leanprover.github.io/papers/lean4.pdf>.